

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»
(Университет ИТМО)**

Факультет безопасности информационных технологий

Образовательная программа: Технологии защиты информации

Направление подготовки(специальность): 10.03.01 Информационная безопасность

О Т Ч Е Т

об учебной практике

Тема задания: Исследование уязвимостей контроля доступа и разработка методики тестирования в формате Attack-Defense CTF

Обучающийся:

студент группы N3447 Чу Ван Доан

Согласовано:

Руководитель практики в ИТМО: Меншиков Александр Алексеевич, доцент ФБИТ, к.т.н., доцент, декан ФБИТ, директор учебно-практического центра - Киберполигон

Ответственный за практику от университета: Еритенко Николай Алексеевич, инженер ФБИТ

Практика пройдена с оценкой _____

Дата 10.02.2026

Санкт-Петербург

2026 г.

СОДЕРЖАНИЕ

Содержание.....	2
Введение.....	3
1 Теоретические и методические основы исследования уязвимостей контроля доступа в Attack-Defense CTF.....	5
1.1 Формат Attack-Defense CTF: раунды, чекер, флаги, статусы (OK/MUMBLE/CORRUPT/DOWN), SLA.....	5
1.2 Контроль доступа в веб-сервисах: аутентификация и авторизация.....	6
1.3 Типовые уязвимости контроля доступа и их последствия для приватных данных....	7
1.4 Принципы построения методики тестирования контроля доступа в условиях A/D...9	
1.4.1 Общие принципы методики.....	9
1.4.2 Критерии pass/fail.....	10
1.4.3 Виды тестов: positive / negative / edge.....	10
2 Разработка методики и тестирование сервиса в рамках Attack-Defense CTF.....	12
2.1 Описание стенда и условий эксперимента.....	12
2.2 Анализ поверхности атаки и точек контроля доступа в сервисе.....	13
2.3 Разработанная методика тестирования: структура тест-кейса и тест-матрица.....	16
2.4 Рекомендации по повышению защищённости и валидация.....	20
2.4.1 Усиление аутентификации и защиты учётных данных.....	20
2.4.2 Защита сессий.....	20
2.4.3 Строгая и однозначная object-level авторизация и изоляция данных.....	21
2.4.4 Корректная обработка ввода/ошибок без утечек.....	21
2.4.5 Валидация защитных изменений (процедура “до/после”).....	21
Заключение.....	23
Список использованных источников.....	24

ВВЕДЕНИЕ

Формат соревнований Attack-Defense CTF представляет собой прикладную модель реальной эксплуатации и защиты информационных систем в условиях непрерывного противодействия. В отличие от классических задач на поиск отдельной уязвимости, в Attack-Defense участникам необходимо одновременно обеспечивать работоспособность сервисов (поддержание статуса ОК и высокого показателя SLA) и противостоять целенаправленным действиям соперников, стремящихся получить доступ к приватным данным пользователей (флагам). В такой среде особенно критичны ошибки, связанные с контролем доступа, поскольку они напрямую приводят к утечкам данных при сохранении внешней «нормальной» работоспособности сервиса.

Актуальность темы обусловлена тем, что логические ошибки авторизации и разграничения прав (например, некорректная проверка владельца ресурса, ошибки обработки идентификаторов объектов, несогласованность проверок в разных эндпоинтах, некорректная работа с сессионными данными) часто являются одной из наиболее распространённых причин компрометации веб-приложений. В условиях Attack-Defense CTF подобные ошибки быстро масштабируются: найденная логическая уязвимость, как правило, воспроизводима на экземплярах сервисов у других команд. При этом защита должна выполняться аккуратно: чрезмерно «жёсткие» меры могут нарушить функциональность, необходимую чекеру, что приводит к статусам MUMBLE/CORRUPT и снижению SLA. Следовательно, требуется системный подход, который сочетает исследование уязвимостей контроля доступа и разработку методики тестирования, ориентированной на особенности Attack-Defense.

Цель работы — исследовать уязвимости контроля доступа в веб-сервисе в условиях соревнований Attack-Defense CTF и разработать практическую методику тестирования, позволяющую выявлять логические ошибки доступа и подтверждать корректность защитных изменений без ухудшения показателей работоспособности (SLA).

Для достижения поставленной цели решаются следующие задачи:

1. Описать формат Attack-Defense CTF и критерии оценки работоспособности сервисов (OK/MUMBLE/CORRUPT/DOWN, SLA), включая роль чекера и флагов.
2. Выполнить анализ исследуемого сервиса: развертывание стенда, сбор диагностических данных, изучение архитектуры, модели данных, механизмов аутентификации/сессий и контрольных точек доступа.
3. Разработать методику тестирования контроля доступа и тест-матрицу (positive/negative/edge), определить критерии pass/fail.

4. Провести тестирование, зафиксировать результаты, сформулировать рекомендации по повышению защищённости, выполнить валидацию изменений и оформить отчёт с приложениями.

Объект исследования — веб-сервис, функционирующий в игровой инфраструктуре Attack-Defense CTF (на примере сервиса типа “Otkritki”/аналогичного приложения с пользовательскими данными).

Предмет исследования — логика контроля доступа (авторизация и разграничение прав) и методы её тестирования в условиях противодействия и требований чекера.

В работе применяются следующие методы: анализ исходного кода и архитектуры (статический анализ), анализ поведения сервиса по журналам событий и сетевому трафику, проектирование тест-сценариев (positive/negative/edge), экспериментальная проверка на развернутом стенде, сравнительный анализ результатов «ожидаемо/фактически», а также валидация защитных изменений повторным тестированием.

Практическая значимость работы заключается в том, что разработанная методика тестирования может быть использована как прикладной инструмент для:

1. выявления логических ошибок контроля доступа, приводящих к утечкам приватных данных (флагов);
2. снижения риска перехода сервиса в статусы MUMBLE/CORRUPT из-за некорректных изменений;
3. повышения стабильности работы сервиса и поддержания высоких значений SLA в Attack-Defense CTF.

Отчёт имеет следующую структуру: в первой главе описывается формат Attack-Defense CTF и критерии оценки сервисов; во второй главе рассматриваются типовые уязвимости контроля доступа; в третьей главе приводится описание стенда и организации диагностики; в четвёртой главе выполняется анализ сервиса и точек контроля доступа; в пятой главе формируется методика тестирования и тест-матрица; в шестой и седьмой главах приводятся результаты тестирования, рекомендации и валидация; в заключении сформулированы основные выводы и направления дальнейшей работы.

1 ТЕОРЕТИЧЕСКИЕ И МЕТОДИЧЕСКИЕ ОСНОВЫ ИССЛЕДОВАНИЯ УЯЗВИМОСТЕЙ КОНТРОЛЯ ДОСТУПА В ATTACK-DEFENSE CTF

1.1 Формат Attack-Defense CTF: раунды, чекер, флаги, статусы (OK/MUMBLE/CORRUPT/DOWN), SLA

Attack-Defense CTF - это соревнование в реальном времени, где каждая команда обслуживает одинаковый набор сетевых сервисов. Команды одновременно защищают свои сервисы (чтобы они работали корректно и не утекали данные) и атакуют сервисы соперников, пытаясь получить из них приватную информацию.

Раунд - это фиксированный интервал времени, по окончании которого инфраструктура соревнования проверяет каждый сервис. Проверку выполняет чекер - программа-эмулятор «реального пользователя», которая по сценарию проверяет функциональность (например: регистрация, вход, создание данных), кладёт новые приватные данные и пытается прочесть ранее сохранённые.

Раунд - это фиксированный интервал времени, по окончании которого инфраструктура соревнования проверяет каждый сервис. Проверку выполняет чекер - программа-эмулятор «реального пользователя», которая по сценарию проверяет функциональность (например: регистрация, вход, создание данных), кладёт новые приватные данные и пытается прочесть ранее сохранённые.

После выполнения раунда чекером сервис получает один из статусов:

1. OK - сервис работает нормально, функциональность доступна.
2. MUMBLE - сервис частично работает, часть функционала «поломана».
3. CORRUPT - приватные данные пользователя (в т.ч. ранее сохранённые флаги) невозможно получить корректно.
4. DOWN - сервис недоступен по сети.

SLA (Service Level Availability) - ключевой показатель стабильности сервиса: доля раундов, в которых сервис имел статус OK, от общего числа раундов.

Чем выше SLA, тем лучше сервис сохраняет работоспособность в условиях атак и изменений.

В ходе соревнования Attack-Defense CTF мы использовали несколько ключевых техник для поиска флага. Во-первых, мы проводили анализ и эксплуатацию уязвимостей в сервисах, используя такие инструменты, как Kali-Linux и распространённые методы эксплуатации уязвимостей. После обнаружения уязвимостей команда быстро устраняла их, чтобы помешать другим командам использовать эти уязвимости. Для ускорения процесса поиска флага мы использовали скрипты, которые автоматизировали атаки и

экономили время. Также мы проводили анализ исходного кода приложений для поиска уязвимых мест или ошибок безопасности, которые можно было бы использовать. В заключение, чёткое распределение задач между членами команды позволило оптимизировать время и повысить эффективность работы, что обеспечило выполнение задач

Round: 185

UP	CORRUPT	MUMBLE	DOWN	CHECK FAILED
----	---------	--------	------	--------------

#	team	score	Zapiski		otkritki	
1	WELCOME 172.25.3.2	7866.63	SLA: 97.30%	● SLA: 96.22%	●	●
			FP: 5612.93	FP: 2500.00		
			■ +346/-52	■ +0/-0		
2	ООО ПЕНЕТРЕЙТОР\$\$ 172.25.1.2	7782.44	SLA: 100.00%	● SLA: 100.00%	●	●
			FP: 5186.11	FP: 2596.32		
			■ +85/-72	■ +10/-8		
3	SSH 172.25.5.2	5075.95	SLA: 99.46%	● SLA: 99.46%	●	●
			FP: 960.94	FP: 4142.59		
			■ +0/-84	■ +31/-0		
4	Ливерпуль 172.25.4.2	3354.33	SLA: 100.00%	● SLA: 98.38%	●	●
			FP: 1143.92	FP: 2246.84		
			■ +0/-73	■ +0/-5		
5	Кабанчики 172.25.2.2	2816.55	SLA: 99.46%	● SLA: 100.00%	●	●
			FP: 1143.73	FP: 1679.00		
			■ +0/-72	■ +0/-15		
6	NPC 172.25.6.2	2747.55	SLA: 98.92%	● SLA: 98.38%	●	●
			FP: 952.36	FP: 1835.24		
			■ +0/-78	■ +0/-13		

Рисунок 1 - Итоговая рейтинговая таблица второго соревнования

1.2 Контроль доступа в веб-сервисах: аутентификация и авторизация

Аутентификация (Authentication) - отвечает на вопрос «Кто ты?»: подтверждение личности перед выдачей сессии (cookie) или токена. Варианты: логин/пароль, cookie-сессия, токены (JWT/API-key), MFA. Важно защищать секреты и сессии от утечек/подмены.

Авторизация (Authorization) - отвечает на вопрос «Что тебе можно?»: проверка прав аутентифицированного пользователя на действие/объект. Реализуется через роли (RBAC), атрибуты (ABAC) или владение (ownership). Проверки должны выполняться на сервере в каждом запросе, а не на фронтенде.

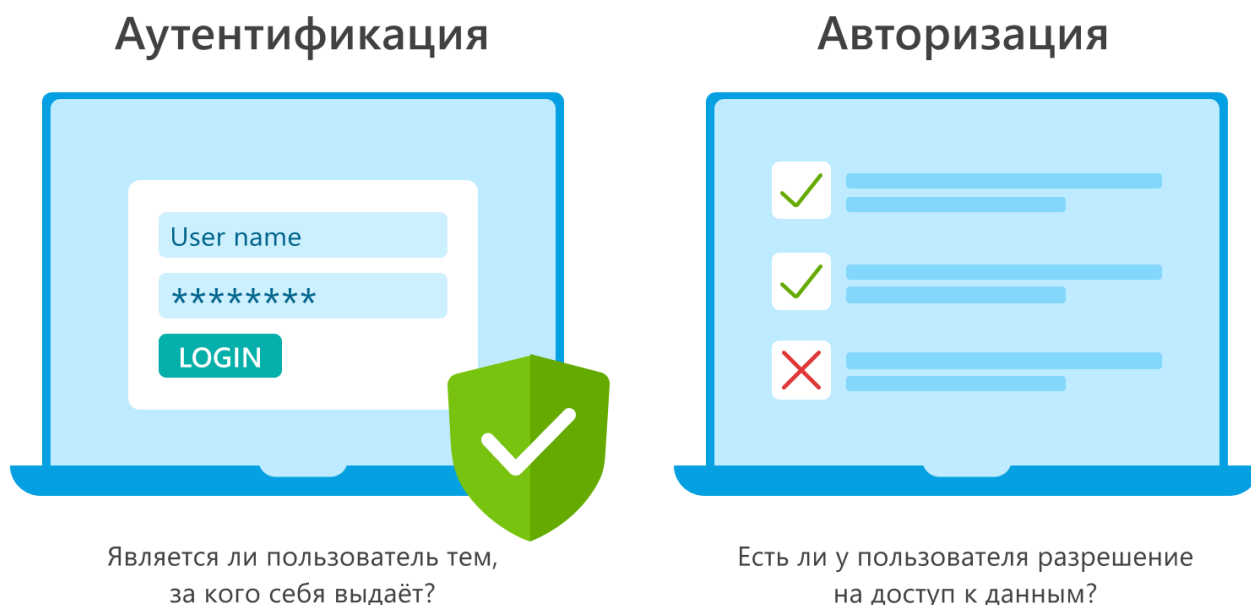


Рисунок 2 - Аутентификация и Авторизация

Пример: пользователь вошёл (аутентификация) -> запрашивает `/api/card?id=123` -> сервер проверяет, имеет ли он право читать именно эту карточку (авторизация).

Типовые ошибки: IDOR/Broken Access Control (доступ по id без проверки), непоследовательные проверки в разных эндпоинтах, логические ошибки условий, проблемы с сессиями, CSRF при cookie-сессиях.

Почему критично в A/D CTF: ошибки авторизации ведут к утечке флагов, а чрезмерные «фиксы» могут сломать сценарии чекера и вызвать MUMBLE/CORRUPT, снижая SLA. Баланс: безопасность + работоспособность.

1.3 Типовые уязвимости контроля доступа и их последствия для частных данных

В веб-сервисах ошибки контроля доступа чаще всего возникают не из-за «сложных» багов, а из-за неправильной логики проверок прав на уровне конкретных объектов данных и операций. В условиях Attack-Defense CTF такие ошибки напрямую приводят к утечкам частных данных (флагов) и потере очков.

- IDOR (Insecure Direct Object Reference) / Broken Access Control по идентификаторам
Суть: доступ к объекту по параметру `id/uuid` без строгой проверки владельца/получателя/роли.

Как проявляется: `"/api/item?id=123"` отдаёт чужие записи при простом переборе.

Последствия: массовое чтение чужих объектов -> кража флагов, резкий рост FP у атакующего и падение у жертвы.

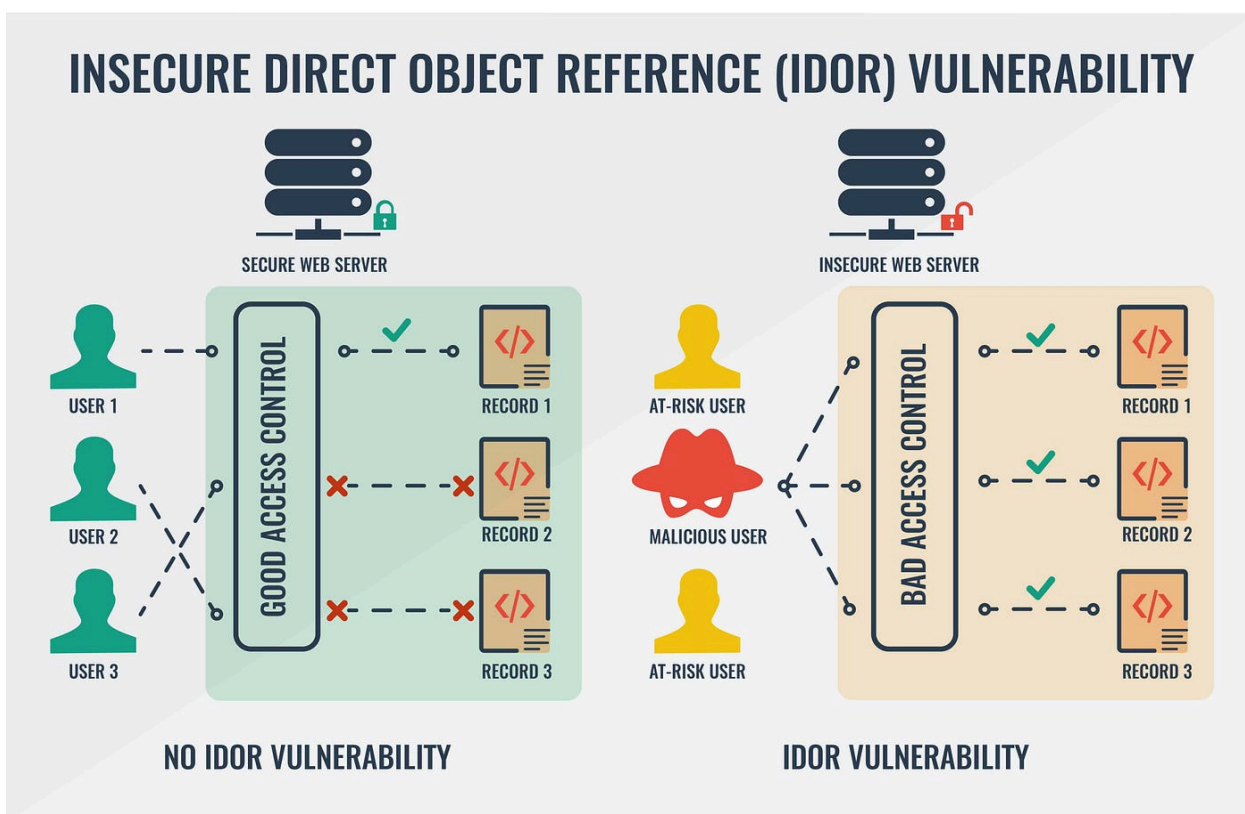


Рисунок 3 - IDOR

- Горизонтальная эскалация прав (между пользователями одного уровня)

Суть: пользователь с обычными правами получает доступ к данным другого обычного пользователя.

Причина: проверка прав построена на неверном условии (например, “совпадает часть имени”, “есть поле в запросе”, “клиент сказал, что он владелец”).

Последствия: утечка флагов без взлома админки; часто незаметно функционально -> сервис остаётся ОК, но данные утекают.

- Вертикальная эскалация прав (обычный пользователь -> админ/модератор)

Суть: обход проверки роли/привилегий и выполнение админ-действий.

Как проявляется: доступ к admin-эндпоинтам без проверки или с “мягкой” проверкой.

Последствия: компрометация всего сервиса: чтение/выгрузка данных, просмотр всех флагов, изменение настроек -> быстрый “слив” флагов.

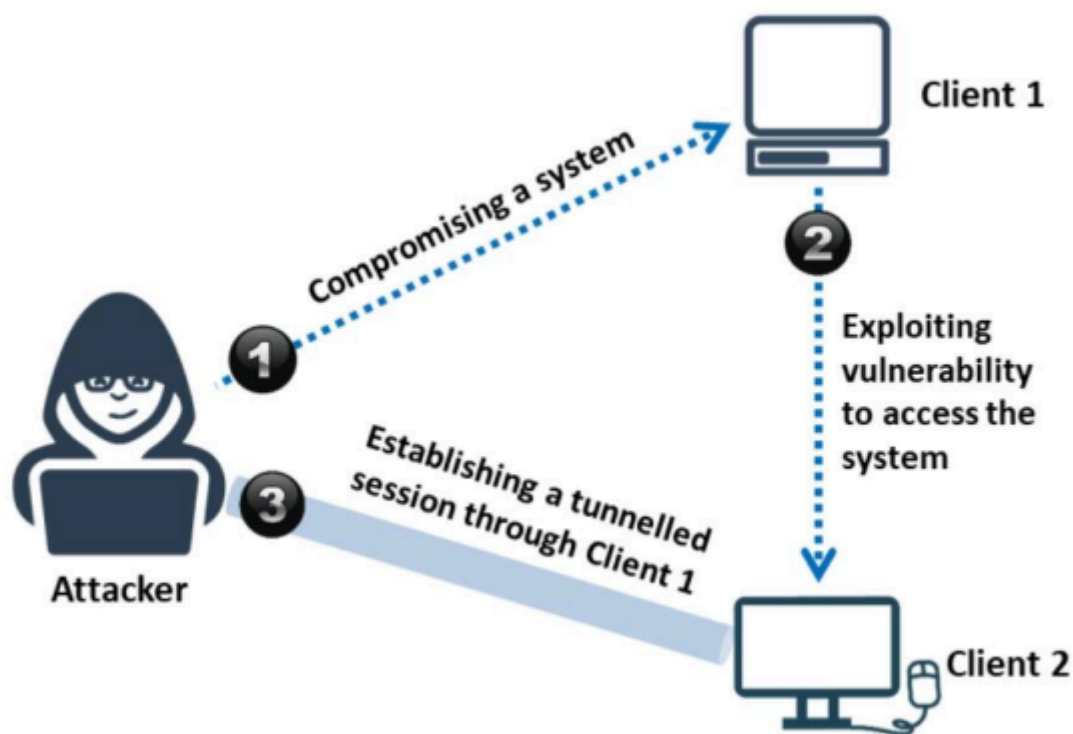


Рисунок 4 - эскалация прав

1.4 Принципы построения методики тестирования контроля доступа в условиях A/D

В условиях A/D методика тестирования контроля доступа должна решать две задачи одновременно: не допустить утечки приватных данных (флагов) и не нарушить сценарии чекера, чтобы не получить MUMBLE/CORRUPT и не просадить SLA.

1.4.1 Общие принципы методики

Сервер - единственная точка доверия: все проверки прав выполняются на сервере для каждого запроса (не “на фронтенде”).

Объектный уровень (object-level) обязателен: права проверяются не “в целом”, а для конкретного объекта (card, note, message) по его идентификатору.

Проверки должны быть единообразны: одинаковые правила доступа должны действовать во всех эндпоинтах (GET/POST/PUT/DELETE).

Тесты недеструктивны по умолчанию: нельзя ломать данные чекера; destructive-тесты (удаление/массовые изменения) — только на отдельном тестовом аккаунте/на тестовых объектах.

Повторяемость: каждый тест имеет чёткие предусловия, шаги и ожидаемый результат; результаты фиксируются (HTTP-код, тело ответа, логи).

1.4.2 Критерии pass/fail

Pass (тест пройден), если:

- Нет утечки данных: неавторизованный пользователь не получает приватные поля/контент, где потенциально есть флаг.
- Нет несанкционированных действий: нельзя создать/изменить/удалить чужие объекты или сменить владельца/роль.
- Корректные ответы сервиса:
 - без сессии -> 401/403 (или эквивалентная ошибка)
 - без прав -> 403 или 404 (допустим “скрывающий” вариант), но без выдачи чужих данных
 - с правами -> 200/201 и корректные данные
- Стабильность: сервис отвечает быстро и стабильно (без ошибок 5xx, без деградации доступности).

Fail (тест провален), если:

- Можно прочитать чужой объект по id (IDOR) или получить данные по “слабому” эндпоинту.
- Можно изменить/удалить чужой объект или подменить чувствительные поля (ownerId, role, isAdmin, и т.п.).
- Проверка прав “плавает”: один эндпоинт защищён, другой - нет.
- Исправление ломает легитимный сценарий (чекер) -> риск MUMBLE/CORRUPT.

1.4.3 Виды тестов: positive / negative / edge

A) Positive tests (позитивные)

Проверяют, что разрешённые действия работают:

- пользователь читает свои объекты;
- получатель видит свои входящие;
- автор видит свои исходящие;
- разрешённые операции (создание/просмотр) выполняются без ошибок.

Цель: не сломать функционал, который нужен чекеру (OK -> SLA).

B) Negative tests (негативные)

Проверяют, что запрещённые действия блокируются:

- пользователь В пытается прочитать объект А по id;
- пользователь В пытается изменить/удалить объект А;
- запрос без сессии/с неправильной сессией;
- попытка обратиться к “админским” функциям без роли.

Цель: доказать отсутствие утечки флагов и обходов авторизации.

С) Edge tests (краевые/неочевидные)

Проверяют обходы через “углы” логики:

- граничные значения параметров: id=0, id=-1, очень большое id, пустое значение, нечисловое;
- альтернативные форматы/кодировки/дубликаты параметров (id=1&id=2), лишние поля (mass assignment);
- ошибки логики условий: подстроки, OR/AND, “совпало частично”, регистр, пробелы;
- поведение при истёкшей/новой сессии, при параллельных запросах.

Цель: поймать именно логические уязвимости контроля доступа, типичные для

A/D.

2 РАЗРАБОТКА МЕТОДИКИ И ТЕСТИРОВАНИЕ СЕРВИСА В РАМКАХ ATTACK-DEFENSE CTF

2.1 Описание стенда и условий эксперимента

Экспериментальная часть работы выполнялась в инфраструктуре Attack-Defense CTF, где каждой команде выделяется собственная игровая машина (vulnbox) с одинаковым набором сервисов. Для команды №2 vulnbox доступен по адресу 172.25.2.2 (шаблон адресации: 172.25.<team>.2). Взаимодействие с проверяющей системой и сдача флагов выполняются через инфраструктуру организаторов (в т.ч. endpoint для отправки флагов и получения attack_data).

Состав стенда

На vulnbox развернуты сервисы, работающие в контейнерной среде Docker. Фактический набор контейнеров и опубликованные порты:

- otkritki-frontend - веб-интерфейс сервиса Otkritki
Порт: 31338/tcp (публикуется на хосте)
- otkritki-backend - backend/API сервиса Otkritki
Порт: 8083/tcp (публикуется на хосте)
- otkritki-db (MySQL) - база данных сервиса Otkritki
Внутренний порт: 3306/tcp (в docker-сети; используется backend'ом)
- zapiski-zapiski - сервис Zapiski
Порт: 5712/tcp (публикуется на хосте)
- zapiski-cleaner (dedcleaner) - вспомогательный компонент обслуживания данных Zapiski
Портов наружу не публикует

Таким образом, стенд включает два прикладных сервиса (Otkritki и Zapiski) и СУБД MySQL, обеспечивающую хранение данных пользователей и объектов приложения.

Сетевые условия и точки доступа

- Внутри vulnbox сервисы доступны как по 127.0.0.1, так и по адресу 172.25.2.2.
- Для Otkritki используется связка frontend (31338) -> backend (8083) -> DB (MySQL).
- Доступ к функциональности backend'а осуществлялся через HTTP API (например, регистрация/логин/операции с объектами), при этом аутентификация реализована через cookie-сессию (cookie session, выдаваемая сервером в ответ на успешную регистрацию/вход).

Условия эксперимента и допущения

- Тестирование проводилось в условиях, типичных для A/D: сервис должен сохранять корректную работу для легитимного пользователя/чекера и одновременно исключать доступ к приватным данным со стороны посторонних.
- Приватные данные в контексте A/D рассматриваются как флаги, размещаемые чекером и доступные только в рамках корректной модели доступа (владелец/получатель/роль).
- Проверки выполнялись на уровне HTTP-запросов и серверной логики контроля доступа: наличие/отсутствие cookie-сессии, разграничение прав между разными пользователями, корректность проверок на “владение” объектами.

Используемые инструменты

- Docker CLI: `docker ps`, `docker logs`, при необходимости `docker exec` — контроль состава стенда и диагностика ошибок.
- `curl` - воспроизводимые запросы к API (фиксация HTTP-кодов, заголовков, cookie, тела ответа).
- Анализ логов backend'a - выявление ошибок обработки запросов, проблем с сессиями и маршрутизацией.
- Tulip (мониторинг трафика) - наблюдение и разбор HTTP-трафика (структура запросов, параметры, cookies, последовательность действий).
- Вспомогательные утилиты ОС: `cat`, `grep`, `ss/netstat` (при необходимости) для уточнения конфигурации.

2.2 Анализ поверхности атаки и точек контроля доступа в сервисе

Поверхность атаки (внешние интерфейсы)

В рамках стенда сервисы доступны по HTTP и образуют две основные точки входа:

- Otkritki-frontend (порт 31338) - веб-интерфейс, инициирующий запросы к API.
- Otkritki-backend (порт 8083) - REST-подобный API, на котором реализованы регистрация, вход и операции с объектами (карточками).

Дополнительно в стенде присутствуют сервис Zapiski (порт 5712) и инфраструктурные контейнеры (БД MySQL, служебный cleaner), которые формируют внутреннюю поверхность атаки внутри docker-сети.

Ключевые эндпоинты backend (по исходному коду routing)

По назначению обработчиков можно выделить следующие группы API:

Аутентификация/сессия

- POST /api/register - регистрация пользователя и выдача сессии (cookie session).
- POST /api/login - вход и выдача сессии.
- POST /api/logout - завершение сессии (инвалидация cookie).
- GET /api/check... - проверка авторизованного состояния (handler CheckAuth).

Работа с карточками (GiftCard)

- GET /api/cards - получение списка карточек пользователя (входящие / последние).
- GET /api/card?id=<id> - получение конкретной карточки по идентификатору.
- POST /api/card / POST /api/add... - создание карточки (handler AddCardPost, доступ ограничен).

Механизм сессий и cookie

Аутентификация в сервисе реализована через cookie-сессию:

- имя cookie: session (const cookieName = "session"),
- хранилище: gorilla/sessions.NewCookieStore(...),
- в сессию записываются ключевые поля:
 - authenticated (bool),
 - gender (string),
 - id (uint).

Точка доверия: наличие корректной cookie session определяет, считается ли пользователь аутентифицированным и какой у него идентификатор/роль (пол).

Критическая особенность реализации: ключ CookieStore задан жёстко в коде ("sixtybytelength"). В условиях A/D это означает высокий риск компрометации механизма сессий при раскрытии исходников: теоретически появляется возможность подделки содержимого cookie и, как следствие, обхода авторизации (имперсонация пользователя).

Middleware и точки проверки доступа

Контроль доступа в backend строится на middleware:

1. AuthMiddleware

Проверяет наличие cookie-сессии и значения:

- authenticated == true,
- id ссылается на существующего пользователя в БД.

Если проверка не проходит - запрос отклоняется (в текущей реализации через abort(...), часто с кодом 404).

2. MaleWiddleWare (ограничение по полу)

Дополнительная проверка для операций создания карточек: разрешено только при `gender == Male`.

Это пример ролевого/атрибутного контроля доступа (ABAC).

3. Локальные проверки в обработчиках (object-level authorization)

- GetCards: идентификатор пользователя берётся из сессии → выборка карточек выполняется по `user.Username`. Таким образом, пользователь получает только “свои” данные (при условии корректности сессии).
- CardPage: выполняется проверка принадлежности карточки пользователю:
 - разрешено, если пользователь является отправителем (`card.From == user.Username`) или получателем.
 - при этом получатель проверяется через `strings.Contains(card.To, user.Username)`, что является потенциально опасной логикой (подстрочное совпадение вместо строгого равенства/парсинга списка получателей). Это может приводить к логическим обходам контроля доступа при определённых значениях `To/Username`.

Существенные точки риска для тестирования (что обязательно проверять)

В рамках методики тестирования контроля доступа для данного сервиса критично покрыть следующие места:

- AuthN/AuthZ на входе: корректность входа/регистрации, невозможность работы с API без валидной сессии.
- Целостность сессии: невозможность подделки/переноса сессии и корректная обработка отсутствующих/некорректных значений `authenticated/id/gender`.
- Object-level access control: попытки чтения чужих объектов через `GET ...?id=...` (класс IDOR/логические ошибки).
- Согласованность проверок: одинаковое поведение для “списка” и “деталей” (например, список скрыт, но детализация по `id` может быть слабее).
- Ролевые ограничения: невозможность выполнения “male-only” действий при другом значении `gender`.
- Поведение при ошибках: отсутствие утечек данных в сообщениях об ошибках и корректные HTTP-ответы без раскрытия приватной информации.

2.3 Разработанная методика тестирования: структура тест-кейса и тест-матрица

```
chu@chu-latitude-5510:~/Desktop/service/service$ export PORT=10001
chu@chu-latitude-5510:~/Desktop/service/service$ docker compose up --build -d
[+] Building 14.1s (17/17) FINISHED
=> [internal] load local bake definitions
=> => reading from stdin 522B
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 651B
=> [internal] load metadata for docker.io/library/ubuntu:latest
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [ 1/10] FROM docker.io/library/ubuntu:latest@sha256:cd1dba651b3080c3686ecf4e3c4220f026b521fb76978881737d24f200828b2b
=> => resolve docker.io/library/ubuntu:latest@sha256:cd1dba651b3080c3686ecf4e3c4220f026b521fb76978881737d24f200828b2b
=> => sha256:a3629ac5b9f4680dc2032439ff2354e73b06aecc2e68f0035a2d7c001c8b4114 29.73MB / 29.73MB
=> => extracting sha256:a3629ac5b9f4680dc2032439ff2354e73b06aecc2e68f0035a2d7c001c8b4114
=> [internal] load build context
=> => transferring context: 2.98kB
=> [ 2/10] RUN apt-get update && apt-get install -y socat
=> [ 3/10] RUN useradd -r -m -d /home/user -s /usr/sbin/nologin user
=> [ 4/10] RUN mkdir /home/user/notes && chown -R root:user /home/user/notes && chmod 770 /home/user/notes
=> [ 5/10] RUN mkdir /home/user/users && chown -R root:user /home/user/users && chmod 770 /home/user/users
=> [ 6/10] RUN mkdir /app && chown -R root:root /app && chmod 755 /app
=> [ 7/10] COPY app/entrypoint.sh app/service.sh /app/
=> [ 8/10] WORKDIR /home/user/
=> [ 9/10] COPY app/entrypoint.sh app/service.sh /app/
=> [10/10] RUN chmod 750 /app/service.sh /app/entrypoint.sh
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:43c88396d00aad2fd25607751101b007e15905593c0dc9fc3ba8befe7da55db5
=> => exporting config sha256:dd197ba7a22b58fb5ffcf902ead4315bab99f2749523a5b9b88c2de0c514934b
=> => exporting attestation manifest sha256:59753361e79a7f83831f5a4505fd337304de3e1f048be106d0c39b7cb22fa221
=> => exporting manifest list sha256:b5f120fb333c094eb26fd4468052aff3d80ac689cf9a375e0218e11099ff3d1f
=> => naming to docker.io/library/service-todoapp:latest
=> => unpacking to docker.io/library/service-todoapp:latest
=> resolving provenance for metadata file
[+] up 3/3
✔ Image service-todoapp Built
✔ Network service_default Created
✔ Container service-todoapp-1 Created
```

Рисунок 5 - Развертывание тестового стенда: сборка и запуск сервиса через Docker Compose

В рамках подготовки стенда для исследования уязвимостей контроля доступа в формате Attack-Defense CTF была выполнена первичная валидация работоспособности сервиса с использованием штатного чекера (режимы CHECK/PUT/GET). На локально развернутом сервисе (адрес 127.0.0.1, порт 10001) последовательно запускались проверки: команда CHECK подтверждала доступность и корректность базового сценария взаимодействия с сервисом (статус ОК, код завершения 101), команда PUT размещала тестовый флаг в хранилище сервиса и возвращала идентификатор доступа (auth_data) для последующей проверки, после чего команда GET по полученному auth_data успешно извлекала ранее записанный флаг и подтверждала его неизменность (также ОК, код 101). Данный этап фиксирует, что сервис функционирует стабильно и соответствует ожиданиям чекера, что позволяет далее проводить тестирование сценариев нарушения контроля доступа (negative/edge-cases) и оценивать результаты исправлений без деградации работоспособности (исключая состояния DOWN/MUMBLE/CORRUPT).


```

(venv) chu@chu-latitude-5510:~/Desktop/service/checker$ export PORT=10001
(venv) chu@chu-latitude-5510:~/Desktop/service/checker$ python checker.py check 127.0.0.1
HibednFPHrMyIJsy
UwJ0cHgfabkdPpmb
uQasIqxQWZjHMuIt
pNsAChTeLYYjmPeX
Exit with code 101
(venv) chu@chu-latitude-5510:~/Desktop/service/checker$ python checker.py get 127.0.0.1 "$AUTH" 'FLAG{demo_flag_123}'
Exit with code 101
(venv) chu@chu-latitude-5510:~/Desktop/service/checker$ export PORT=10001
(venv) chu@chu-latitude-5510:~/Desktop/service/checker$ python checker.py check 127.0.0.1
UEhSMKB0hiPXtbxI
DEMXJSfBiFYWqLDk
OqbqTqeXhyecjswY
maDhSnRKgVHXtgcl
Exit with code 101
(venv) chu@chu-latitude-5510:~/Desktop/service/checker$ python checker.py put 127.0.0.1 1 'FLAG{demo_flag_123}'
rChRQJJQvKBLKQo:LJkgNjixEBvsMZNx
Exit with code 101
(venv) chu@chu-latitude-5510:~/Desktop/service/checker$ AUTH=$(python checker.py put 127.0.0.1 1 'FLAG{demo_flag_123}' | grep -v 'Exit with code' | head -n 1)
Exit with code 101
(venv) chu@chu-latitude-5510:~/Desktop/service/checker$ python checker.py get 127.0.0.1 "$AUTH" 'FLAG{demo_flag_123}'
Exit with code 101
(venv) chu@chu-latitude-5510:~/Desktop/service/checker$

```

Рисунок 6 - Валидация работоспособности сервиса в формате Attack-Defense CTF:

успешное прохождение чекера (CHECK/PUT/GET, статус OK, код 101)

Тест-кейс AC-E1 (Edge): попытка получить чужие заметки/флаги через инъекцию в поле содержимого заметки.

Ожидание (PASS): пользователь В не видит данные пользователя А, флаг не раскрывается.

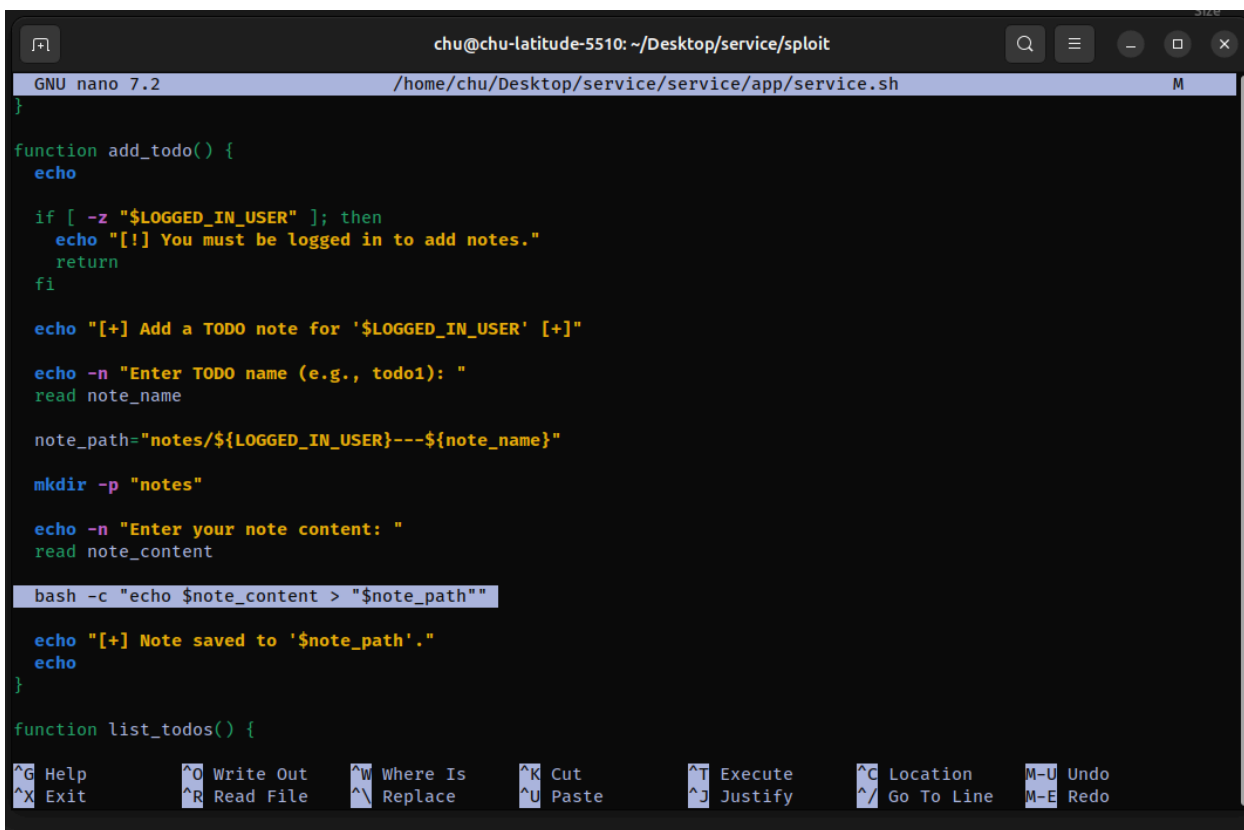
Факт (FAIL): в выводе sploit.py отображаются чужие заметки и значения FLAG{...} (утечка флага).

```

(venv) chu@chu-latitude-5510:~/Desktop/service/sploit$ python sploit.py 127.0.0.1 10001 | tee ../evidence_sploit.txt
[x] Opening connection to 127.0.0.1 on port 10001
[x] Opening connection to 127.0.0.1 on port 10001: Trying 127.0.0.1
[+] Opening connection to 127.0.0.1 on port 10001: Done
[x] Receiving all data
[x] Receiving all data: 0B
[x] Receiving all data: 1B
[x] Receiving all data: 313B
[+] Receiving all data: Done (313B)
[*] Closed connection to 127.0.0.1 port 10001
b"\n[+] TODO notes for 'lbBSqliFpxvAQSnY' [+] \n-----\nTODO: LmxUvjzFLmgcXciq\npNsAChTeLY
YjmPeX FLAG{demo_flag_123} FLAG{demo_flag_123} maDhSnRKgVHXtgcl YEUqDpXHjitaccnW FLAG{demo_flag_123} FLAG{demo_flag_
123} FLAG{demo_flag_123}\n-----\n\nPress Enter to continue...\n"
(venv) chu@chu-latitude-5510:~/Desktop/service/sploit$

```

Рисунок 7 - Результат эксплуатации (до исправления)



The screenshot shows a terminal window with the title bar "chu@chu-latitude-5510: ~/Desktop/service/sploit". The nano editor is open to the file "/home/chu/Desktop/service/service/app/service.sh". The code in the editor is as follows:

```
}

function add_todo() {
    echo

    if [ -z "$LOGGED_IN_USER" ]; then
        echo "[!] You must be logged in to add notes."
        return
    fi

    echo "[+] Add a TODO note for '$LOGGED_IN_USER' [+]"

    echo -n "Enter TODO name (e.g., todo1): "
    read note_name

    note_path="notes/${LOGGED_IN_USER}---${note_name}"

    mkdir -p "notes"

    echo -n "Enter your note content: "
    read note_content

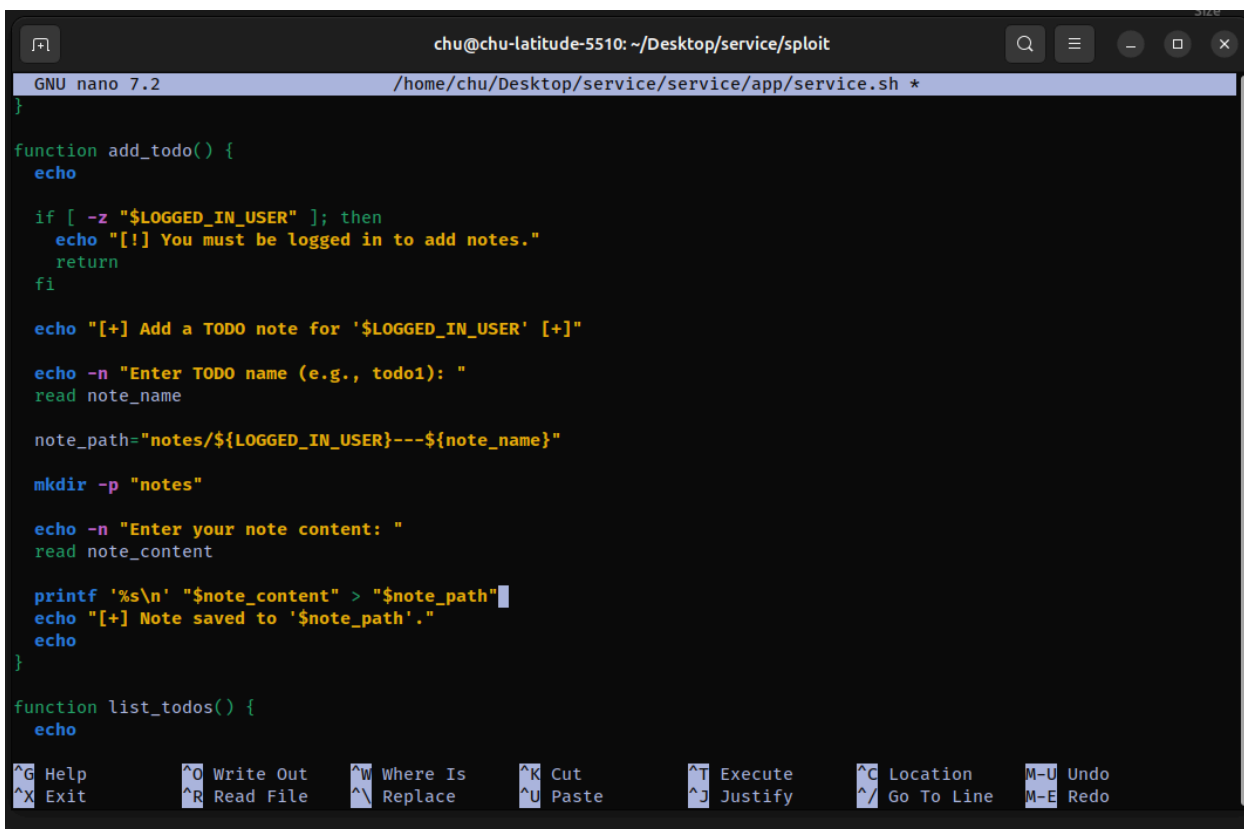
    bash -c "echo $note_content > \"$note_path\""

    echo "[+] Note saved to '$note_path'."
    echo
}

function list_todos() {
```

The bottom status bar of the nano editor shows various keyboard shortcuts: ^G Help, ^X Exit, ^O Write Out, ^R Read File, ^W Where Is, ^N Replace, ^K Cut, ^U Paste, ^T Execute, ^J Justify, ^C Location, ^_ Go To Line, M-U Undo, and M-E Redo.

Рисунок 8 - Уязвимый фрагмент кода в service.sh



The screenshot shows the same terminal window as Figure 8, but with the corrected code in the nano editor. The code is as follows:

```
}

function add_todo() {
    echo

    if [ -z "$LOGGED_IN_USER" ]; then
        echo "[!] You must be logged in to add notes."
        return
    fi

    echo "[+] Add a TODO note for '$LOGGED_IN_USER' [+]"

    echo -n "Enter TODO name (e.g., todo1): "
    read note_name

    note_path="notes/${LOGGED_IN_USER}---${note_name}"

    mkdir -p "notes"

    echo -n "Enter your note content: "
    read note_content

    printf '%s\n' "$note_content" > "$note_path"
    echo "[+] Note saved to '$note_path'."
    echo
}

function list_todos() {
    echo
```

The bottom status bar of the nano editor is identical to the one in Figure 8.

Рисунок 9 — Исправленный фрагмент кода в service.sh

```

(venv) chu@chu-latitude-5510:~/Desktop/service/service$ cd ../checker
(venv) chu@chu-latitude-5510:~/Desktop/service/checker$ export PORT=10001
(venv) chu@chu-latitude-5510:~/Desktop/service/checker$ python checker.py check 127.0.0.1
pWqareNXkcwgMPYg
iuHuDqUWxEtqpTOb
aUwzVVDeLMWDjDLI
FgAQFqhSPSCzmxRV
Exit with code 101
(venv) chu@chu-latitude-5510:~/Desktop/service/checker$ AUTH=$(python checker.py put 127.0.0.1 1 'FLAG{demo_flag_123}' | grep -v 'Exit with code' | head -n 1)
Exit with code 101
(venv) chu@chu-latitude-5510:~/Desktop/service/checker$ python checker.py get 127.0.0.1 "$AUTH" 'FLAG{demo_flag_123}'
Exit with code 101
(venv) chu@chu-latitude-5510:~/Desktop/service/checker$ cd ../sploit
(venv) chu@chu-latitude-5510:~/Desktop/service/sploit$ python exploit.py 127.0.0.1 10001 | tee ../evidence_sploit_after_fix.txt
[x] Opening connection to 127.0.0.1 on port 10001
[x] Opening connection to 127.0.0.1 on port 10001: Trying 127.0.0.1
[+] Opening connection to 127.0.0.1 on port 10001: Done
[x] Receiving all data
[x] Receiving all data: 0B
[x] Receiving all data: 1B
[x] Receiving all data: 179B
[+] Receiving all data: Done (179B)
[*] Closed connection to 127.0.0.1 port 10001
b"\n[+] TODO notes for 'frZBDgCA0tdNIMXX' [+] \n-----\nTODO: PwhWTINyUzsaloEu\n$(cat ./notes/*)\n-----\n\nPress Enter to continue...\n"
(venv) chu@chu-latitude-5510:~/Desktop/service/sploit$

```

Рисунок 10 — Валидация исправления в формате Attack-Defense CTF

В ходе исследования уязвимостей контроля доступа был выполнен edge-тест AC-E1 на обход изоляции пользовательских данных через внедрение команд в поле содержимого заметки. В качестве полезной нагрузки использовалась конструкция вида `$(cat ./notes/*)`, которая при некорректной обработке ввода может приводить к чтению файлов с заметками других пользователей и, как следствие, к раскрытию флагов.

До исправления тест AC-E1 завершался неуспешно: вывод скрипта эксплуатации (`sploit.py`) содержал значения `FLAG{...}`, что подтверждало выполнение внедрённой команды и чтение файлов `notes/*`, принадлежащих другим пользователям. Данный факт трактуется как нарушение контроля доступа на уровне объектов (утечка чужих данных/флагов) и соответствует критическому воздействию в условиях Attack-Defense (компрометация флагов).

После исправления уязвимость была устранена путём запрета выполнения пользовательского ввода как команды оболочки (ввод сохраняется как обычный текст без интерпретации). Повторный запуск AC-E1 показал, что полезная нагрузка отображается в заметке только в текстовом виде и не приводит к раскрытию `FLAG{...}`. При этом регрессионная проверка работоспособности сервиса штатным чекером (режимы CHECK/PUT/GET) продолжает завершаться со статусом ОК (код 101), что подтверждает сохранение корректного поведения сервиса и отсутствие деградации SLA/сценариев чекера.

AC-E1 (Edge) — Попытка чтения чужих заметок/флагов через command injection в содержимом заметки

- Предусловия: сервис запущен; чекер успешно выполнил PUT и сохранил флаг в заметке пользователя А.
- Шаги: пользователь В создаёт заметку с содержимым `$(cat ./notes/*)` и запрашивает список заметок (используется `sploit.py`).
- Ожидаемый результат (PASS): внедрённая команда не выполняется; вывод не содержит `FLAG{...}`, чужие заметки недоступны.
- Фактический результат до исправления: FAIL — в выводе присутствуют `FLAG{...}` (утечка данных/флагов).
- Фактический результат после исправления: PASS — полезная нагрузка сохраняется как текст, `FLAG{...}` не раскрывается; чекер CHECK/PUT/GET остаётся OK (101).

2.4 Рекомендации по повышению защищённости и валидация

2.4.1 Усиление аутентификации и защиты учётных данных

Для снижения риска захвата аккаунтов и дальнейших атак на приватные данные рекомендуется обеспечить строгую проверку учётных данных и безопасное хранение паролей. В общем случае пароли должны храниться в виде криптографического хэша (например, Argon2/bcrypt), а при входе — сравниваться с хэшем, а не проверяться косвенно по существованию пользователя

Для исследуемого TODO-сервиса дополнительно целесообразно:

- ограничить формат username/password (allowlist), чтобы исключить спецсимволы, влияющие на логику/файловые пути;
- добавить базовую защиту от перебора (rate-limit/задержка после N ошибок);
- исключить вывод диагностических данных (путей, внутренних ошибок), которые могут помочь атакующему.

2.4.2 Защита сессий

В сервисах с cookie-сессиями рекомендуется хранить ключи подписи/шифрования вне кода (в переменных окружения), использовать отдельные ключи (подпись + шифрование) и корректно настраивать параметры cookie (HttpOnly/SameSite/Secure/MaxAge). Для текущего TODO-сервиса (CLI/без cookie) данный пункт трактуется как общий принцип: любые секреты (ключи, пароли, токены) не должны быть “захардкожены” и должны быть заменяемыми без пересборки.

2.4.3 Строгая и однозначная object-level авторизация и изоляция данных

В A/D критично, чтобы доступ проверялся на уровне конкретного объекта, а правила были единообразны и не допускали “логических обходов. Для исключения обходов рекомендуется отказаться от “нестрогих” проверок (например, подстрочных

совпадений) в пользу строгого равенства/корректного парсинга и привязки прав к идентификаторам, а не к строковым полям.

Для вашего TODO-сервиса это выражается в следующих мерах:

- жёсткая изоляция файлов заметок по пользователям (строгая схема имени файла + запрет ../, *, пробелов и т.п.);
- исключение glob-обходов при перечислении заметок (без небезопасного раскрытия файлов, не принадлежащих пользователю);
- запрет исполнения пользовательского ввода в командной оболочке

2.4.4 Корректная обработка ввода/ошибок без утечек

Ключевое правило: не возвращать содержимое чужого объекта ни при каких ошибках и разделять случаи “нет сессии/нет прав/объект не существует” корректными кодами/сообщениями.

В рамках выполненного исследования была выявлена утечка флагов через выполнение пользовательского ввода как команды оболочки при сохранении заметки (command injection). Рекомендуемое исправление — сохранять введённый текст “как есть”, без интерпретации shell (например, через `printf '%s\n' "$note_content" > "$note_path"`), а также применять allowlist-валидацию для имени заметки и пользователя. Это устраняет возможность чтения `notes/*` и предотвращает раскрытие `FLAG{...}`.

2.4.5 Валидация защитных изменений (процедура “до/после”)

Валидация выполнялась повторным прогоном ключевых сценариев по методике (positive/negative/edge) с фиксацией артефактов и контролем критериев pass/fail:

1. Регрессия чекера (positive): последовательный запуск `CHECK` → `PUT` → `GET`. Успешность подтверждается статусом ОК (в вашем стенде — exit code 101) и тем, что флаг, записанный на этапе `PUT`, корректно извлекается на этапе `GET`. Цель шага — убедиться, что исправление не ломает легитимный сценарий (иначе риск MUMBLE/CORRUPT).
2. Проверка устранения утечки (edge AC-E1): повторный запуск `sploit.py` с полезной нагрузкой `$(cat ./notes/*)`.
 - До исправления: в выводе присутствовали `FLAG{...}`, что означало чтение чужих заметок/флагов и провал теста (Fail по критерию “есть утечка данных”).
 - После исправления: полезная нагрузка сохраняется как текст и не исполняется, `FLAG{...}` в выводе отсутствует — тест пройден (Pass).

3. Контроль стабильности: сервис продолжает отвечать предсказуемо, без аварийных ошибок/зависаний, что соответствует критерию стабильности и недопущения деградации доступности.

Таким образом, рекомендации по усилению защищённости были подтверждены практической валидацией: устранена утечка флагов, при этом базовые сценарии чекера сохранены и сервис остаётся в статусе ОК, что соответствует целям методики тестирования в формате Attack-Defense CTF

ЗАКЛЮЧЕНИЕ

В рамках работы была исследована проблематика уязвимостей контроля доступа и разработана практическая методика тестирования в формате Attack-Defense CTF, ориентированная на сохранение работоспособности сервиса и сценариев чекера при одновременном поиске критических логических ошибок (утечек флагов/данных). Предложенный подход основан на разделении тестирования на три уровня: (1) регрессионная проверка чекером (CHECK/PUT/GET) для контроля статусов OK/MUMBLE/CORRUPT/DOWN и соблюдения SLA, (2) негативные тесты для подтверждения запрета несанкционированных действий, (3) edge-сценарии для выявления обходов логики и нестандартных векторов атак.

На экспериментальном стенде на базе предоставленного сервиса был подтверждён класс уязвимости, приводящий к нарушению контроля доступа на уровне объектов: через обработку пользовательского ввода небезопасным способом происходило выполнение команд оболочки, что позволяло читать содержимое файлов с заметками других пользователей и извлекать флаги. Данный результат демонстрирует типовую для Attack-Defense ситуацию, когда дефект в обработке данных фактически трансформируется в нарушение изоляции объектов и компрометацию флагов.

Для повышения защищённости сервиса выполнено исправление, исключаящее интерпретацию пользовательского ввода оболочкой, а также предложены дополнительные меры укрепления: allowlist-валидация входных параметров, предотвращение обходов через файловые пути/шаблоны, единообразные проверки прав доступа и безопасная обработка ошибок без утечек информации. Эффективность исправлений подтверждена повторной валидацией: после внесения изменений чекер стабильно проходит сценарии CHECK/PUT/GET со статусом OK, а повторный запуск эксплуатационного скрипта не приводит к раскрытию FLAG{...}.

Таким образом, работа достигла поставленной цели: разработана и апробирована методика тестирования контроля доступа в условиях Attack-Defense CTF, обеспечивающая воспроизводимость результатов, прозрачные критерии pass/fail и корректную проверку исправлений без деградации работоспособности сервиса. В качестве направлений дальнейшего развития целесообразны расширение тест-матрицы (в т.ч. на сценарии изменения/удаления объектов и модели ролей), автоматизация регрессионного запуска тестов по раундам, а также внедрение базового мониторинга и журналирования событий безопасности для ускорения реагирования в соревновательной среде.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Attack-Defense CTF: что это такое и как в это играть – Текст: электронный // CBS CTF: сайтсайт. – URL: <https://cbsctf.ru/ad> (дата обращения: 10.02.2026).
2. Authentication Cheat Sheet – Текст: электронный // OWASP Cheat Sheet Series: сайтсайт. – URL: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html (дата обращения: 10.02.2026).
3. Авторизация. Определение и методы – Текст: электронный // eKassir: сайтсайт. – URL: <https://ekassir.com/blog/avtorizacziya-opredelenie-i-metody/> (дата обращения: 10.02.2026).
4. IDOR (Insecure Direct Object Reference) – Текст: электронный // Medium: сайтсайт. – URL: <https://medium.com/@vulnerable19/idor-insecure-direct-object-reference-cb4a54fd8a0a> (дата обращения: 10.02.2026).
5. CEHCEN Module 6 – Phần 5: Leo thang đặc quyền – Текст: электронный // SinhVienCNTT: сайтсайт. – URL: <https://sinhviencontt.net/ceh-tieng-viet-leo-thang-dac-quyen-2236> (дата обращения: 10.02.2026).